

Disjoint Set Union (Union Find)

Dynamic Islands Application

This is one of the most important applications of Disjoint Set Union (Union Find).

The main idea is:

Instead of finding islands from scratch after every operation using DFS/BFS ($O(N \times M)$), we dynamically merge islands using DSU in almost $O(1)$ time.

Problem

Initially the entire grid is water.

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Then operations are performed.

Example

$(0, 0)$, $(1, 1)$, $(1, 0)$, $(0, 1)$

After every operation we have to return:

"How many islands currently exist?"

Brute Force

Every operation:



Count islands

Time

- K operations
- Each DFS = $O(N*M)$
- Total = $O(K*N*M)$

This is far too slow.

Better Observation

Suppose we add only one land cell. What can happen? Only three possibilities.

Case 1

No neighbour is land.

0	0	0
0	X	0
0	0	0

New island created.

Count++

Case 2

One neighbour is land.

0	1	0
0	X	0
0	0	0

X simply joins that island.

Count remains same.

Case 3

Multiple neighbouring islands exist.

1	0	1
0	X	0
0	0	0

Case 3 Example Deep Dive:

Before: Island A, Island B

After adding X:



Now: Two islands become one. Count--

If there are three different neighbouring islands A, B, C ↓ A-X-B-X-C ↓ One island. Then Count decreases twice.



So we only need to know: **Which neighbouring cells belong to different islands?**
That is exactly what DSU does.

Why DSU?

Each land cell belongs to one component. Whenever two adjacent lands touch, **Merge them**.

DSU supports:

- Find parent
- Union

Both nearly $O(1)$. Hence every operation is almost constant time.

Mapping 2D Grid to 1D

DSU works on numbers. Grid `4 x 5` becomes:

0	1	2	3	4
5	6	7	8	9
10	11	12	13	14
15	16	17	18	19

Formula:

$$nodeNumber = row \times m + col$$

Example ($m = 5$):

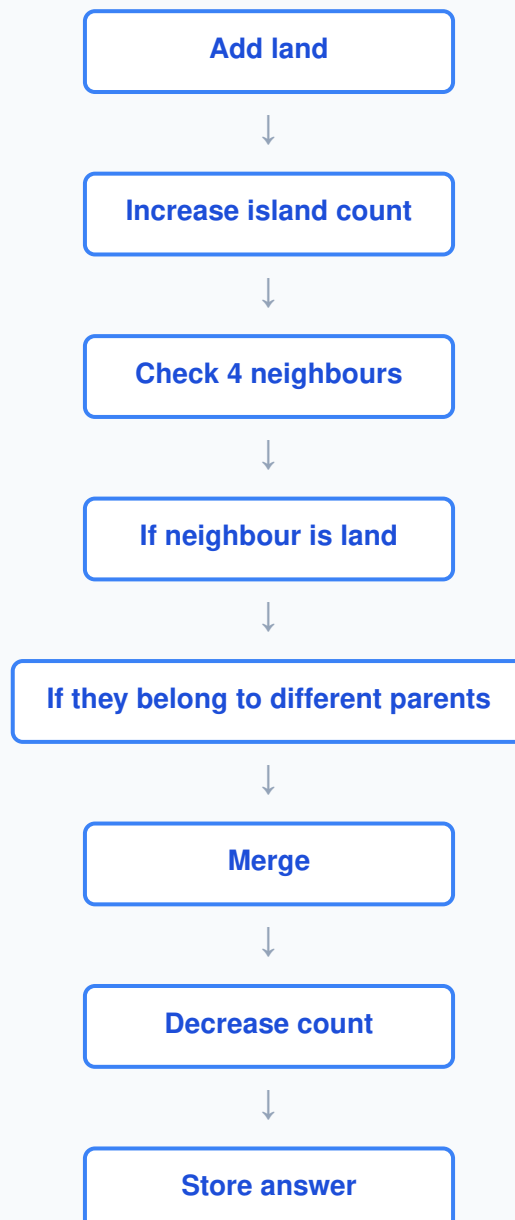
- $(0, 0) \rightarrow 0 \times 5 + 0 = 0$
- $(0, 3) \rightarrow 0 \times 5 + 3 = 3$
- $(1, 2) \rightarrow 1 \times 5 + 2 = 7$
- $(3, 4) \rightarrow 3 \times 5 + 4 = 19$

So every cell has one DSU node.

Data Structures

- `DisjointSet ds(n*m)` : Stores connectivity.
- `vis[n][m]` : Stores whether land already exists. (0 = water, 1 = land)
- `cnt` : Current number of islands.
- `ans` : Answer after every operation.

Algorithm



Now let's understand every line

Step 1

```
DisjointSet ds(n * m);
```

Every grid cell becomes one DSU node. For `4 x 5`, Total nodes = 20.

Step 2

```
int vis[n][m];  
memset(vis,0,sizeof vis);
```

Initially everything is water.

Step 3

```
cnt=0;
```

No islands initially.

Process Every Operation

Operation 1: (0,0)

Mark land: `vis[0][0]=1;`

Increase: `cnt=1`

Now check neighbours: Up (Invalid), Right (Water), Down (Water), Left (Invalid). No merges.

Answer: 1

DSU: `0` (Only one component)

L	.	.	.	.
.	.	.	.	.
.	.	.	.	.
.	.	.	.	.

Operation 2: Again (0,0)

Already land. `if(vis[row][col]==1)` is True. Nothing changes.

Answer: 1

Operation 3: (1,1)

Increase: `cnt=2`

Neighbours: All water.

Answer: 2

DSU Components: `0` and `6`

L	.	.	.	.
.	L	.	.	.
.	.	.	.	.
.	.	.	.	.

0

6

Operation 4: (1,0)

Initially **cnt=3** because we added new land. Now check neighbours.

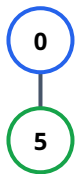
Neighbour Up: (0,0) (Land)

Current node (1,0) $\rightarrow 1 \times 5 + 0 = 5$

Neighbour (0,0) $\rightarrow 0$

Different parents (5, 0). Merge. **cnt=2**

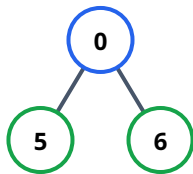
L	.	.	.	.
L	L	.	.	.
.	.	.	.	.
.	.	.	.	.



Neighbour Right: (1,1) (node=6)

Current parent **0**, Neighbour parent **6**. Different. Merge again. **cnt=1**

Now entire island connected:



Remaining neighbours: Nothing. **Answer: 1**

Notice: Initially 3 islands. After two unions $3 \rightarrow 2 \rightarrow 1$. Exactly correct.

Operation 5: (0,1)

Initially `cnt=2` (New land added). Now check neighbours.

Neighbour (0,0): Same island. Parent `0`, Current `1`. Union.

`cnt=1`

Next neighbour (1,1): Its parent is already `0`. Current parent after previous union is `0`. Both parents same. **No merge.**

L	L	.	.	.
L	L	.	.	.
.	.	.	.	.
.	.	.	.	.

This is why this line exists:

```
if(ds.findUPar(nodeNo)!=ds.findUPar(adjNodeNo))
```

Otherwise we'd reduce island count twice incorrectly. Without this condition: $2 \rightarrow$ Union with left $\rightarrow 1 \rightarrow$ Union with down $\rightarrow 0$ ❌ Wrong.

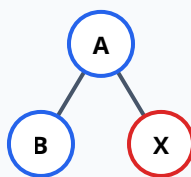
With DSU: Left parent=0, Down parent=0, Already same. Skip. **Correct answer: 1**

Diagram of Union

Initially A, B:

After adding X: `A--X--B`

Union: `parent(B)=A`. Now Single island:



Understanding This Part

```
int nodeNo = row*m+col;
```

Suppose `row=2`, `col=3`, `m=5`.

$2 \times 5 + 3 = 13$. Cell `(2,3)` becomes **Node 13**.

Neighbour `(2,2)` $\rightarrow 2 \times 5 + 2 = 12$. Union **13** and **12**.

Four Directions

```
dr[] = {-1,0,1,0};  
dc[] = {0,1,0,-1};
```

These pairs represent:

Index	dr	dc	Direction
0	-1	0	Up
1	0	1	Right
2	1	0	Down
3	0	-1	Left

Loop `for(int ind=0;ind<4;ind++)` checks every neighbour.

Why cnt++ First?

Suppose new land appears. Initially `.` We assume New island created. So `cnt++`.

Then if neighbours connect: Merge, `cnt--`.

This is much simpler than first trying to predict whether a new island is formed.

Complete Flowchart





Time Complexity

Let:

- K = number of operations
- $N \times M$ = grid size

For each operation:

- Marking land: $O(1)$
- Checking 4 neighbors: $4 \times O(1)$
- Each find/union: $O(\alpha(N \times M))$, where α (inverse Ackermann function) is effectively constant (less than 5 in practice).

So the total complexity is:

$$O(K \times \alpha(NM)) \approx O(K)$$

Space complexity:

- DSU arrays (parent, size, rank): $O(NM)$
- vis array: $O(NM)$

Total:

$$O(NM)$$

Intuition to Remember for Interviews

When asked "Why DSU instead of DFS/BFS?", explain it like this:

"Each operation changes only one cell. Running DFS/BFS over the entire grid after every update repeats a lot of work. DSU maintains connected components incrementally. When a new land cell is added, it initially forms its own island (`cnt++`). Then we inspect its four neighbors. Every neighboring land that belongs to a different component is merged using DSU (union), and each successful merge reduces the island count (`cnt--`). This lets us answer each operation in almost constant time instead of recomputing all islands from scratch."